

Arbres de décision, Random Forests et XGBoost– Notes de révision détaillées

Préparé pour révision d’entretien — version détaillée

8 octobre 2026

Résumé

Ces notes condensées donnent un niveau technique et algorithmique suffisant pour les entretiens quant et pour la pratique en production. Elles couvrent : CART, critères de split, pruning, bagging / Random Forest (avec preuve courte de la réduction de variance), gradient boosting, formulation exacte de XGBoost (avec dérivations de la forme fermée des poids de feuille et du gain), différences d’implémentation (LightGBM, CatBoost), gestion des valeurs manquantes et catégories, interprétabilité (importances, SHAP), et réglages pratiques. Le texte est en français et vise à être lu avant un entretien.

Table des matières

1	Introduction	2
2	CART : Construction d’un arbre de décision	2
2.1	Problème et idée générale	2
2.2	Critères d’impureté	2
2.3	Algorithme (greedy CART)	3
2.4	Critères d’arrêt et élagage (pruning)	3
2.4.1	Critères d’arrêt	3
3	Random Forests	4
3.1	Principe	4
3.2	Pourquoi ça réduit la variance ?	4
3.3	Out-of-bag (OOB) estimate	5
3.4	Hyperparamètres importants	5
4	Boosting et Gradient Boosting	5
4.1	Idée générale	5
4.2	Algorithme de gradient boosting (schéma)	5
5	XGBoost : formulation détaillée	5
5.1	Notations	5
5.2	Objectif régularisé et poids feuille	6
5.3	Gain d’un split	6
5.4	Résumé des hyperparamètres XGBoost	6
6	Implémentations et différences pratiques	6
6.1	XGBoost vs LightGBM vs CatBoost	6
6.2	Méthodes de recherche de split	7

7	Gestion des valeurs manquantes / catégories / jamais-vus	7
7.1	Valeurs manquantes	7
7.2	Catégorielles	7
7.3	Valeurs jamais-vues	7
8	Interprétabilité	7
8.1	Types d'importance de features	7
8.2	SHAP et TreeSHAP	7
9	LightGBM : principes et fonctionnement	7
9.0.1	Principe du Gradient Boosting	8
9.0.2	Spécificités de LightGBM	8
9.0.3	Récapitulatif du processus d'entraînement	9
10	Preuves courtes et formules à connaître	9
10.1	Réduction de variance par bagging	9
10.2	Poids optimal des feuilles (XGBoost)	9
11	Réglages pratiques et recettes pour la production	9
11.1	Règles pratiques	9
11.2	Surveillance / garanties	10
12	Exemples concrets en finance	10
13	Questions d'entretien et réponses prêtes (formulations courtes)	10
14	Lectures et références courtes	10
15	Annexe : formules et cheatsheet rapide (à mémoriser)	10

1 Introduction

Objectif : fournir une base solide et formelle sur ce que font réellement les arbres et les algorithmes dérivés (Random Forest, Gradient Boosting, XGBoost) — pas seulement l'intuition. On inclut définitions, formules, pseudo-algorithmes et petites preuves utiles en entretien.

2 CART : Construction d'un arbre de décision

2.1 Problème et idée générale

On dispose d'un jeu de données $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ avec $x_i \in \mathbb{R}^p$. Un arbre partitionne récursivement l'espace des x en régions disjointes $R_1, \dots, R_M$ et prédit par une valeur constante c_m sur chaque région R_m :

$$\hat{f}(x) = \sum_{m=1}^M c_m \mathbf{1}_{\{x \in R_m\}}.$$

La construction est gloutonne : à chaque nœud on choisit la variable et le seuil qui maximisent le gain d'impureté.

2.2 Critères d'impureté

— **Classification** :

— *Gini impurity* :

$$G = 1 - \sum_{k=1}^K p_k^2,$$

avec p_k la proportion de la classe k dans le nœud. Le gain d'un split est

$$\Delta G = G_{parent} - \frac{n_L}{n} G_L - \frac{n_R}{n} G_R.$$

— *Entropie* :

$$H = - \sum_{k=1}^K p_k \log p_k.$$

— **Régression** : impureté définie par la variance locale (ou RSS) :

$$I(\text{noeud}) = \frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2, \quad \bar{y} = \frac{1}{n} \sum_i y_i.$$

Gain = variance(parent) – moyenne pondérée des variances des enfants.

2.3 Algorithme (greedy CART)

CART-greedy(D, stopping-criteria):

```
if stopping-criteria satisfied: return leaf with prediction c = average(y)
for each feature j:
    for each threshold s in sorted(unique(x_j)):
        compute impurity gain for split x_j <= s
choose (j*, s*) with max gain
split D into D_left, D_right
node.left = CART-greedy(D_left, ...)
node.right = CART-greedy(D_right, ...)
return node
```

2.4 Critères d'arrêt et élagage (pruning)

La croissance d'un arbre de décision peut théoriquement se poursuivre jusqu'à ce que chaque feuille contienne une seule observation. Cependant, cela conduit à un modèle sur-appris (*overfitting*). Pour éviter cela, on introduit des **critères d'arrêt** pendant la construction, et/ou une étape d'**élagage** (*post-pruning*) après coup.

2.4.1 Critères d'arrêt

Ces critères contrôlent la profondeur et la complexité de l'arbre pendant son apprentissage :

- **max_depth** : profondeur maximale de l'arbre. *Plus la profondeur est grande, plus le modèle est flexible.* Une valeur trop élevée conduit à un fort sur-apprentissage ; une valeur trop faible, à un sous-apprentissage.
- **min_samples_split** : nombre minimal d'observations requis dans un nœud pour qu'il puisse être divisé. Si le nombre d'échantillons est inférieur à ce seuil, le nœud devient une feuille. *Augmenter ce paramètre rend l'arbre plus robuste mais moins précis localement.*
- **min_samples_leaf** : nombre minimal d'observations autorisé dans une feuille terminale. Cela garantit que chaque région terminale repose sur un minimum de données, ce qui stabilise les prédictions (notamment en régression). *Typiquement, une valeur de 5–10 est souvent utilisée en pratique.*

- **max_leaf_nodes** : nombre maximal de feuilles autorisées dans l'arbre. L'entraînement s'arrête lorsque ce nombre est atteint, même si d'autres splits pourraient encore réduire l'impureté. *Ce paramètre agit directement sur la parcimonie du modèle.*
- **min_impurity_decrease** : seuil minimal de gain d'impureté requis pour effectuer un split. On ne divise un nœud que si

$$\Delta I = I_{\text{parent}} - \frac{n_L}{n} I_L - \frac{n_R}{n} I_R \geq \text{min_impurity_decrease}.$$

Cela évite des divisions insignifiantes qui n'apportent pas de véritable amélioration.

Ces paramètres servent à régulariser la croissance de l'arbre pour limiter la variance du modèle.

Algorithme du cost-complexity pruning. L'objectif est de simplifier un arbre T en supprimant les sous-arbres dont la contribution à la réduction d'impureté est faible par rapport à leur complexité.

[H] [1] **Entrée** : arbre complet T_0 , paramètre de pénalité $\alpha \geq 0$. **Initialisation** : $T \leftarrow T_0$. l'arbre T contient des nœuds internes chaque nœud interne t de T Calculer l'erreur du sous-arbre T_t : $R(T_t) = \sum_{l \in T_t} p_l I_l$ Calculer l'erreur si t devient une feuille : $R(t) = p_t I_t$ Calculer le coût critique :

$$\alpha_t = \frac{R(t) - R(T_t)}{|T_t| - 1}$$

Trouver le plus petit α_t ; couper le sous-arbre correspondant. Obtenir un nouvel arbre T_{next} . **Sortie** : une séquence d'arbres

$$T_0 \supset T_1 \supset \dots \supset T_K$$

associée à une suite croissante de valeurs $\alpha_0 < \alpha_1 < \dots < \alpha_K$. La sélection finale de α^* (et donc de T_{α^*}) se fait par validation croisée.

Interprétation. À chaque itération, on identifie le sous-arbre dont la suppression provoque la plus petite augmentation d'erreur par feuille supprimée. Autrement dit, on retire en priorité les branches dont la *réduction d'impureté pondérée* est la moins significative.

3 Random Forests

3.1 Principe

Random Forest = bagging (bootstrap aggregation) + sous-échantillonnage des features au split (**max_features**). On construit B arbres indépendants sur $\mathcal{D}^{(b)}$ bootstrapés et on agrège : moyenne (régression) ou vote (classification).

3.2 Pourquoi ça réduit la variance ?

Supposons $T_1, \dots, T_B$ estimateurs (arbres) identiquement distribués avec $\text{Var}(T_i) = \sigma^2$ et covariance $\text{Cov}(T_i, T_j) = \rho\sigma^2$ pour $i \neq j$. Alors :

$$\begin{aligned} \text{Var}\left(\frac{1}{B} \sum_{i=1}^B T_i\right) &= \frac{1}{B^2} (B\sigma^2 + B(B-1)\rho\sigma^2) \\ &= \sigma^2 \left(\rho + \frac{1-\rho}{B}\right). \end{aligned}$$

Deux cas intéressants : si ρ est petit (arbres peu corrélés) alors l'agrégation baisse fortement la variance ; si ρ est proche de 1, l'effet est limité. Le sous-échantillonnage de features vise à réduire ρ .

3.3 Out-of-bag (OOB) estimate

Pour chaque observation, on peut utiliser uniquement les arbres qui ne l'ont pas incluse dans leur bootstrap pour estimer l'erreur générale sans CV. C'est une estimation pratique et peu coûteuse.

3.4 Hyperparamètres importants

Paramètre	Rôle
n_estimators	nombre d'arbres (B)
max_features	features aléatoires considérées au split (diminue corrélation)
max_depth	limite la profondeur (contrôle complexité)
min_samples_leaf	taille minimale feuille (évite overfit)
oob_score	estimation OOB de la généralisation

4 Boosting et Gradient Boosting

4.1 Idée générale

On construit un modèle additif :

$$\hat{f}(x) = \sum_{t=1}^T \eta h_t(x),$$

où chaque h_t (typiquement un arbre faible) est entraîné pour corriger l'erreur résiduelle du modèle courant. Le boosting baisse le biais ; le bagging baisse la variance.

4.2 Algorithme de gradient boosting (schéma)

```
Initialize f0(x) = argmin_c sum_i L(y_i, c) # ex: mean(y) for MSE
for t in 1..T:
    compute gradients g_i = dL(y_i, f_{t-1}(x_i))/df
    optionally compute Hessians h_i = d2L/d f2
    fit a regression tree h_t(x) to target -g_i (or approx quad objective)
    choose leaf weights w_j (closed-form if 2nd order used)
    f_t = f_{t-1} + eta * h_t
return f_T
```

5 XGBoost : formulation détaillée

XGBoost introduit une objective régularisée et utilise une approximation de Taylor seconde-ordre pour optimiser les poids des feuilles.

5.1 Notations

Soit $\ell(y, \hat{y})$ la loss (ex : MSE ou log-loss). À l'itération t , pour chaque observation on note

$$g_i = \left. \frac{\partial \ell(y_i, \hat{y})}{\partial \hat{y}} \right|_{\hat{y}=f_{t-1}(x_i)}, \quad h_i = \left. \frac{\partial^2 \ell(y_i, \hat{y})}{\partial \hat{y}^2} \right|_{\hat{y}=f_{t-1}(x_i)}.$$

Si un nœud j regroupe les indices I_j , définissons $G_j = \sum_{i \in I_j} g_i$ et $H_j = \sum_{i \in I_j} h_i$.

5.2 Objectif régularisé et poids feuille

La perte approchée au second ordre pour un arbre de feuilles $\{w_j\}$ est

$$\mathcal{L}^{(t)} \approx \sum_j (G_j w_j + \frac{1}{2} H_j w_j^2) + \sum_j \Omega(w_j), \quad \Omega(w) = \alpha |w| + \frac{1}{2} \lambda w^2.$$

Si $\alpha = 0$ (pas de L1), on dérive et on obtient le poids optimal d'une feuille :

$$w_j^* = -\frac{G_j}{H_j + \lambda}.$$

Avec $\alpha > 0$ (L1 + L2) on applique un soft-thresholding $S(G, \alpha) = \text{sign}(G) \max(|G| - \alpha, 0)$ et

$$w_j^* = -\frac{S(G_j, \alpha)}{H_j + \lambda}.$$

5.3 Gain d'un split

Pour un split divisant un nœud en deux L et R , le gain théorique est :

$$\text{Gain} = \frac{1}{2} \left(\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G^2}{H + \lambda} \right) - \gamma,$$

où $G = G_L + G_R$ et $H = H_L + H_R$, et γ est le coût minimal pour créer un split (`min_split_loss`).

Interprétation Le terme $\frac{G_j^2}{H_j + \lambda}$ mesure la qualité d'une feuille : grande magnitude des gradients (forte direction de descente) normalisée par l'incertitude (H_j) et la régularisation λ .

5.4 Résumé des hyperparamètres XGBoost

- `eta` (`learning_rate`) : facteur d'amortissement de chaque arbre.
- `n_estimators` : nombre d'arbres T .
- `max_depth` : profondeur max d'un arbre.
- `min_child_weight` : somme minimale des h_i dans une feuille (empêche feuilles peu informatives).
- `subsample` : fraction des exemples pris pour entraîner chaque arbre.
- `colsample_bytree` / `colsample_bylevel` : fraction des features au niveau de l'arbre / du split.
- `lambda` (L2), `alpha` (L1) : régularisation des poids des feuilles.
- `gamma` : gain minimal requis pour accepter un split.

6 Implémentations et différences pratiques

6.1 XGBoost vs LightGBM vs CatBoost

- **XGBoost** : implémentation robuste, utilise formulation seconde-ordre et régularisation fine. Propose exact et histogram split.
- **LightGBM** : optimisé pour la vitesse et la mémoire ; croissance *leaf-wise* (plutôt que niveau-wise) et histogram-based splitting. Avantage : plus rapide et souvent meilleure performance, inconvénient : plus susceptible d'overfitter si `max_depth` non borné.
- **CatBoost** : conçu pour jouer correctement avec des features catégorielles via des encodages de type target-encoding ordonnés (ordered target statistics) et boosting ordonné pour réduire le leakage. Bon pour datasets avec beaucoup de catégories.

6.2 Méthodes de recherche de split

- **Exact greedy** : on considère tous les seuils — coûteux en temps pour grandes dimensions.
- **Histogram-based** : on binne chaque feature (ex. 256 bins) et on calcule les gains sur bins — très efficace (LightGBM, XGBoost optionnel).

7 Gestion des valeurs manquantes / catégories / jamais-vus

7.1 Valeurs manquantes

La plupart des implémentations apprennent une "direction par défaut" : lors du calcul du split optimal, on choisit pour les valeurs manquantes la branche (gauche/droite) qui minimise l'objectif ; cette direction est stockée et utilisée en inférence.

7.2 Catégorielles

Stratégies classiques : one-hot (si faible cardinalité), ordinal (prudence), target-encoding (risque de leakage si mal fait). CatBoost propose un encodage ordonné sécurisé et des combinaisons de catégories.

7.3 Valeurs jamais-vues

Un arbre n'extrapole pas : une valeur numérique hors range est simplement dispatchée selon les comparaisons $x_j \leq s$ et aboutit à une feuille existante ; la prédiction est la valeur de cette feuille (piecewise-constant). Pour catégorielles jamais vues, selon l'implémentation on peut avoir : erreur, direction "missing" par défaut, ou fallback via encodage.

8 Interprétabilité

8.1 Types d'importance de features

- **Gain** : somme des gains (améliorations de la loss) pour tous les splits utilisant la feature. Biaisée vers variables avec nombreuses valeurs distinctes.
- **Frequency / Count** : combien de fois la feature est utilisée pour split.
- **Cover** : somme des observations couvertes par les splits de la feature.
- **Permutation importance** : on perturbe la feature et on mesure la dégradation de performance — plus robuste mais coûteux.

8.2 SHAP et TreeSHAP

SHAP fournit des valeurs de Shapley à chaque prédiction ; pour arbres, TreeSHAP calcule ces valeurs exactement en temps polynomial. SHAP permet d'obtenir des explications locales et de quantifier interactions entre variables.

9 LightGBM : principes et fonctionnement

Idée générale. LightGBM (*Light Gradient Boosting Machine*) est un algorithme de **gradient boosting sur arbres de décision**, développé par Microsoft. Il repose sur le principe du **boosting additif** : on construit itérativement un ensemble d'arbres faibles, chacun corrigeant les erreurs du modèle précédent.

9.0.1 Principe du Gradient Boosting

On cherche une fonction $F(x)$ minimisant une perte empirique

$$L = \sum_{i=1}^n \ell(y_i, F(x_i)).$$

Le modèle est construit de manière additive :

$$F_M(x) = \sum_{m=1}^M \nu f_m(x),$$

où :

- $f_m(x)$ est un arbre de décision appris à l'itération m ,
- $\nu \in (0, 1]$ est le *learning rate*.

À chaque étape, on approxime la direction du **gradient négatif** de la fonction de perte :

$$g_i^{(m)} = \left. \frac{\partial \ell(y_i, F(x_i))}{\partial F(x_i)} \right|_{F=F_{m-1}}, \quad h_i^{(m)} = \left. \frac{\partial^2 \ell(y_i, F(x_i))}{\partial F(x_i)^2} \right|_{F=F_{m-1}}.$$

L'arbre f_m est ajusté pour prédire $-g_i^{(m)}$ (ou en pratique une approximation du ratio $-g_i/h_i$ si l'on utilise une expansion de Taylor du second ordre, comme dans XGBoost et LightGBM).

9.0.2 Spécificités de LightGBM

LightGBM introduit plusieurs améliorations majeures par rapport au Gradient Boosting classique :

1. Croissance *leaf-wise* (avec profondeur maximale). Contrairement à XGBoost (croissance *level-wise*), LightGBM choisit à chaque étape la **feuille** dont le gain d'information est maximal :

$$\Delta I = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma,$$

où G et H sont les sommes des gradients et hessiennes dans chaque sous-région, et λ, γ des paramètres de régularisation. Cette stratégie donne souvent une meilleure précision pour un même nombre d'arbres, mais peut sur-apprendre si la profondeur n'est pas limitée (`max_depth`).

2. Approximation par histogrammes. Les valeurs continues des variables sont discrétisées en **bins** (histogrammes). Lors de la recherche du meilleur split, LightGBM parcourt les bins au lieu des valeurs brutes, ce qui réduit :

- la complexité de calcul ($O(\text{nb_bins})$ au lieu de $O(n)$),
- la mémoire utilisée.

3. Gradient-based One-Side Sampling (GOSS). LightGBM accélère encore l'entraînement en échantillonnant préférentiellement les points ayant de **grands gradients** (donc mal prédits). Les observations à faible gradient sont sous-échantillonnées, mais leur contribution est rééchelonnée pour conserver une estimation non biaisée du gradient global.

4. Exclusive Feature Bundling (EFB). Lorsque certaines variables sont mutuellement exclusives (jamais non-nulles simultanément), LightGBM les **regroupe en une seule feature binaire combinée**. Cela réduit la dimensionnalité sans perte d'information et accélère les calculs.

9.0.3 Récapitulatif du processus d'entraînement

1. Initialiser $F_0(x)$ par une constante (souvent la moyenne des y_i).
2. Pour $m = 1, \dots, M$:
 - (a) Calculer les gradients $g_i^{(m)}$ et hessiennes $h_i^{(m)}$.
 - (b) Construire un arbre $f_m(x)$:
 - à croissance *leaf-wise*,
 - en utilisant la recherche de split sur histogrammes,
 - éventuellement avec GOSS et EFB.
 - (c) Mettre à jour le modèle :
$$F_m(x) = F_{m-1}(x) + \nu f_m(x).$$
3. Retourner $F_M(x)$ comme modèle final.

—

En résumé. LightGBM est un gradient boosting rapide et efficace, combinant :

- une croissance **leaf-wise** optimisée,
- des **histogrammes** pour réduire la complexité,
- des techniques d'**échantillonnage intelligent** (GOSS, EFB),

tout en conservant le cadre théorique solide du boosting de gradients de seconde ordre.

10 Preuves courtes et formules à connaître

10.1 Réduction de variance par bagging

Démonstration déjà fournie plus haut (section Random Forests). Citer la formule

$$\text{Var} \left(\frac{1}{B} \sum T_i \right) = \sigma^2 \left(\rho + \frac{1-\rho}{B} \right).$$

Argument d'entretien : expliquer que diminuer la corrélation ρ (via `max_features`) est crucial.

10.2 Poids optimal des feuilles (XGBoost)

Partir de l'approximation quadratique et dériver la condition du premier ordre donne

$$w^* = -\frac{G}{H + \lambda} \quad (\text{cas } \alpha = 0),$$

et le soft-threshold si $\alpha > 0$.

11 Réglages pratiques et recettes pour la production

11.1 Règles pratiques

- Commencer conservateur : `max_depth=3-6`, `learning_rate=0.01-0.1`, `subsample=0.6-0.9`, `colsample=0.5-0.8`.
- Utiliser Early stopping (validation set) pour éviter overfitting lorsque `n_estimators` est grand.
- Pour datasets bruyants / peu de données : Random Forest ou arbres peu profonds ; pour datasets riches et signal clair : XGBoost/LightGBM mieux adaptés.
- Gérer correctement les features catégorielles (CatBoost si bcp de catégories).

11.2 Surveillance / garanties

- OOB / CV pour estimer la généralisation.
- Mesures de robustesse : test sur périodes hors-échantillon (en finance, backtests rolling).
- Attention aux leakage (target encoding, feature-engineering temporelle) : utiliser encodages ordonnés ou folds temporels.

12 Exemples concrets en finance

- **Signal trading faible SNR** : commencer RF (stabilité, variance faible), features factorielles (momentum, volatility) ; architecture simple pour éviter overfit.
- **Défaut prediction / scoring** : si dataset peu volumineux et bruité, RF baseline ; si interactions complexes entre variables de risque, XGBoost avec régularisation.
- **Stacking** : apprentissage d'une représentation (PCA/NN) suivi d'un ensemble d'arbres est une pratique courante quand les features sont corrélées.

13 Questions d'entretien et réponses prêtes (formulations courtes)

1. Pourquoi Random Forest plutôt que XGBoost ? *RF = baseline robuste quand peu de données / fort bruit (bagging baisse variance). XGBoost = mieux si données plus abondantes et besoin d'interactions fines (boosting baisse le biais) ; mais XGBoost nécessite régularisation forte.*
2. Comment XGBoost régularise-t-il ? *Par L1/L2 sur poids de feuilles (alpha/lambda), shrinkage (eta), subsample/colsample, min_child_weight, et gamma (gain minimal pour splitter).*
3. Que se passe-t-il si on a une valeur jamais vue ? *L'arbre n'extrapole pas : on suit les comparaisons et on tombe sur une feuille existante, la prédiction reste piecewise-constant.*

14 Lectures et références courtes

- Breiman, Friedman, Olshen, Stone — *Classification and Regression Trees* (CART).
- Chen Guestrin — *XGBoost : A Scalable Tree Boosting System* (paper).
- Hastie, Tibshirani, Friedman — *The Elements of Statistical Learning* (chapters sur tree-based methods).
- Documentation XGBoost / LightGBM / CatBoost pour détails d'implémentation.

15 Annexe : formules et cheatsheet rapide (à mémoriser)

- Gini : $G = 1 - \sum p_k^2$.
- Variance bagging : $\text{Var}(\bar{T}) = \sigma^2(\rho + (1 - \rho)/B)$.
- Poids feuille XGBoost (L2) : $w^* = -G/(H + \lambda)$.
- Gain split XGBoost : $\frac{1}{2}(G_L^2/(H_L + \lambda) + G_R^2/(H_R + \lambda) - G^2/(H + \lambda)) - \gamma$.

Si tu veux que j'ajoute :

- exemples de code (scikit-learn XGBoost) mis dans des snippets LaTeX ;
- un PDF compilé prêt à imprimer ;
- un petit exercice guidé (notebook) pour manipuler OOB et visualiser effet des hyperparamètres.